Heapsort

- Given **BuildHeap()**, a sorting algorithm is easily constructed:
  - Maximum element is at A[1]
  - Discard by swapping with element at A[n]
    - Decrement heap_size[A]
    - A[n] now contains correct value
  - Restore heap property at A[1] by calling **Heapify()**
  - Repeat, always swapping A[1] for A[heap_size(A)]

Heapsort

```
Heapsort(A)
{
     BuildHeap(A);
     for (i = length(A) downto 2)
     {
          Swap(A[1], A[i]);
          heap_size(A) -= 1;
          Heapify(A, 1);
     }
}
```

Analyzing Heapsort

- The call to **BuildHeap()** takes $O(n)$ time
- Each of the $n - 1$ calls to **Heapify()** takes $O(\log n)$ time

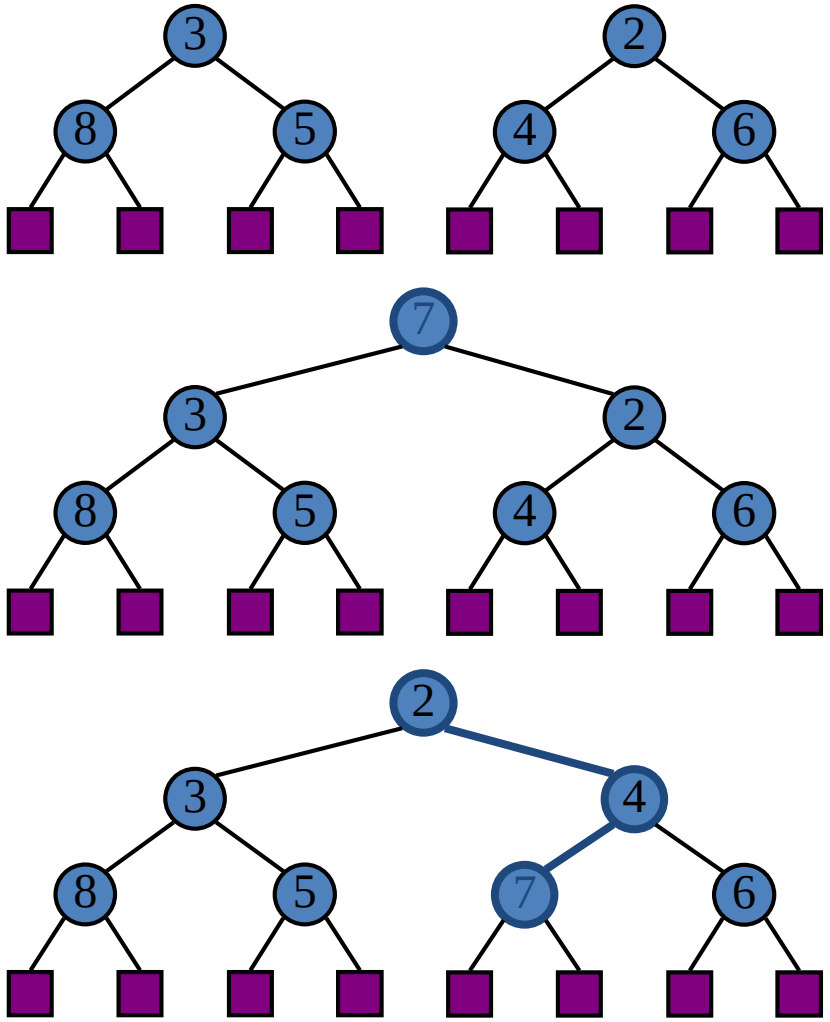
- Thus, the total time taken by **HeapSort()**
  $= O(n) + (n - 1)\, O(\log n)$
  $= O(n) + O(n \log n)$
  $= O(n \log n)$

Summary of Sorting Algorithms

Algorithm	Time	Notes
selection-sort	$O(n^2)$	◆ in-place ◆ slow (good for small inputs)
insertion-sort	$O(n^2)$	◆ in-place ◆ slow (good for small inputs)
quick-sort	$O(n \log n)$	◆ in-place, randomized ◆ fastest (good for large inputs)
heap-sort	$O(n \log n)$	◆ in-place ◆ fast (good for large inputs)
merge-sort	$O(n \log n)$	◆ sequential data access ◆ fast (good for huge inputs)

Merging Two Heaps (MinHeap)

- We are given two heaps and a key ***k***
- We create a new heap with the root node storing ***k*** and with the two heaps as subtrees
- We perform downheap to restore the heap-order property



Priority Queue ADT

- A priority queue stores a collection of items
- An item is a pair (key, element)
- Main methods:
  - insertItem(k, o)
    inserts an item with key k and element o
  - removeMin()
    removes the item with the smallest key
- Additional methods:
  - minKey(k, o)
    returns, but does not remove, the smallest key of an item
  - minElement()
    returns, but does not remove, the element of an item with smallest key
  - size(), isEmpty()
- Applications:
  - Printing
  - Auctions
  - Stock market

Priority Queue

- **Insert(S, x)** inserts the element x into set S
- **Maximum(S)** returns the element of S with the maximum key
- **ExtractMax(S)** removes and returns the element of S with the maximum key
- *One could implement these operations using a heap.*